topic_7_clean_hexagonal_architecture

Theory, Edge Cases & Code Examples

- ✓ What is Hexagonal Architecture?
- ✓ Simple Explanation: Power Outlet Analogy
- ✓ The Problem with Traditional Layered Architecture
- ✓ Hexagonal Architecture Solution
- ✓ Key Concepts

→ *Swipe to learn* →

topic_7_clean_hexagonal_architecture

Theory

📌 What is Hexagonal Architecture?

Hexagonal Architecture (also called **Ports and Adapters** or **Clean Architecture**) is a design pattern that places your **business logic at the center** and makes it **independent** of external concerns (databases, UI, frameworks).

Key Idea: Business logic should not depend on external details. External details (database, UI, web framework) depend on business logic through **interfaces (ports)**.

---

topic_7_clean_hexagonal_architecture

Theory

📌 Simple Explanation: Power Outlet Analogy

Imagine a laptop:

Traditional Architecture	Hexagonal Architecture
Laptop hardwired to wall outlet	**Laptop with power adapter**
Can only use in one country	Use anywhere with different adapters
Can't test without electricity	Can test with battery
Tight coupling	**Loose coupling**

Hexagonal: Your laptop (business logic) doesn't care about the power source (database, UI). It just needs a **port** (interface), and you can plug in different **adapters** (implementations).

---

topic_7_clean_hexagonal_architecture

Theory

📌 The Problem with Traditional Layered Architecture

Traditional Layered Architecture:

```
┌─────────────────┐
│  UI Layer       │
└─────────────────┘
        ↓ depends on
┌─────────────────┐
│  Business Layer │
└─────────────────┘
        ↓ depends on
┌─────────────────┐
│  Data Layer     │
└─────────────────┘
        ↓ depends on
┌─────────────────┐
│  Database       │
└─────────────────┘
```

Problem: Business logic depends on concrete database implementation!

```
class OrderService {  // Business Layer
    MySQLOrderRepository repository;  // ← Depends on MySQL!

    void placeOrder(Order order) {
        repository.save(order);  // ← Can't test without MySQL
    }
};
```

Issues:
- ❌ Can't test business logic without database
- ❌ Can't switch databases (MySQL → PostgreSQL) easily
- ❌ Business logic coupled to infrastructure

---

topic_7_clean_hexagonal_architecture

Theory

📌 Hexagonal Architecture Solution (1/2)

Hexagonal Architecture:

```
                ┌──────────────────┐
                │   UI Adapter     │ (Web, Mobile, CLI)
                └──────────────────┘
                        ↓
                ┌──────────────────┐
                │   Primary Port   │ (Interface)
                └──────────────────┘
                        ↓
        ┌──────────────────────────────────┐
        │                                  │
        │        BUSINESS LOGIC            │
        │        (Core Domain)             │
        │                                  │
        └──────────────────────────────────┘
                        ↓
                ┌──────────────────┐
                │ Secondary Port   │ (Interface)
                └──────────────────┘
                        ↓
                ┌──────────────────┐
                │  DB Adapter      │ (MySQL, Postgres, InMemory)
                └──────────────────┘
```

Key Change: Business logic depends on **interfaces (ports)**, not concrete implementations!

```
// Port (Interface)
class IOrderRepository {
public:
    virtual void save(const Order& order) = 0;
    virtual ~IOrderRepository() = default;
};

// Business Logic
class OrderService {
    IOrderRepository* repository;  // ← Depends on interface!

public:
// ...continues on next slide
```

📌 Hexagonal Architecture Solution (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    OrderService(IOrderRepository* repo) : repository(repo) {}

    void placeOrder(Order order) {
        repository->save(order);  // ← Can test with mock!
    }
};

// Adapter (Implementation)
class MySQLOrderRepository : public IOrderRepository {
    void save(const Order& order) override {
        // MySQL-specific code
    }
};

class InMemoryOrderRepository : public IOrderRepository {
    void save(const Order& order) override {
        // In-memory for testing
    }
};
```

Benefits:
- ✅ Test business logic with mock/in-memory repository
- ✅ Swap MySQL → PostgreSQL by changing adapter
- ✅ Business logic independent of infrastructure

---

topic_7_clean_hexagonal_architecture

Theory

📌 Key Concepts

topic_7_clean_hexagonal_architecture

Theory

📌 1. Hexagon (Business Logic Core) (1/2)

The **hexagon** represents your **core business logic** (domain models, use cases, business rules).

Characteristics:
- ✅ Contains **pure business logic** (no database, no UI)
- ✅ Has **no dependencies** on external frameworks
- ✅ Defines **interfaces (ports)** for external interactions

Example:

```
// Core Business Logic
class Order {
    int id;
    double total;
    std::string status;  // "pending", "completed", "cancelled"

public:
    void confirm() {
        if (status != "pending") {
            throw std::logic_error("Can only confirm pending orders");
        }
        status = "completed";
    }
};

class OrderService {
    IOrderRepository* repository;  // Port (interface)
    IPaymentGateway* paymentGateway;  // Port (interface)

public:
    void placeOrder(Order order, const PaymentInfo& payment) {
        // Business rule: Validate order
        if (order.total <= 0) {
            throw std::invalid_argument("Order total must be positive");
        }

        // Business rule: Charge payment
        paymentGateway->charge(payment, order.total);

        // Business rule: Confirm order
// ...continues on next slide
```

topic_7_clean_hexagonal_architecture

Theory

📌 1. Hexagon (Business Logic Core) (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        order.confirm();

        // Save order
        repository->save(order);
    }
};
```

Note: `OrderService` has NO idea about MySQL, REST APIs, or UI frameworks!

---

topic_7_clean_hexagonal_architecture

Theory

📌 2. Ports (Interfaces)

Ports are **interfaces** that define how the business logic interacts with the outside world.

Two types:

Primary Ports (Driving / Inbound): How external world **uses** your business logic.
- Example: REST API, CLI commands, GUI buttons

```
// Primary Port: Use Cases
class IOrderService {
public:
    virtual void placeOrder(Order order) = 0;
    virtual Order getOrder(int orderId) = 0;
    virtual ~IOrderService() = default;
};
```

Secondary Ports (Driven / Outbound): How business logic **depends on** external systems.
- Example: Database, email service, payment gateway

```
// Secondary Port: Repository
class IOrderRepository {
public:
    virtual void save(const Order& order) = 0;
    virtual Order getById(int id) = 0;
    virtual ~IOrderRepository() = default;
};

// Secondary Port: Payment Gateway
class IPaymentGateway {
public:
    virtual void charge(const PaymentInfo& payment, double amount) = 0;
    virtual ~IPaymentGateway() = default;
};
```

---

topic_7_clean_hexagonal_architecture

Theory

📌 3. Adapters (Implementations) (1/2)

Adapters are **concrete implementations** of ports. They handle external technology details.

Primary Adapters: Translate external requests into business logic calls.

```
// Primary Adapter: REST API Controller
class OrderController {
    IOrderService* orderService;  // Uses Primary Port

public:
    void handlePlaceOrderRequest(HttpRequest request) {
        // Parse HTTP request
        Order order = parseOrderFromJSON(request.body);

        // Call business logic
        orderService->placeOrder(order);

        // Return HTTP response
        return HttpResponse{200, "{\"status\": \"success\"}"};
    }
};
```

Secondary Adapters: Implement secondary ports with specific technology.

```
// Secondary Adapter: MySQL Repository
class MySQLOrderRepository : public IOrderRepository {
    DatabaseConnection db;

public:
    void save(const Order& order) override {
        std::string query = "INSERT INTO orders (total, status) VALUES ("
                            + std::to_string(order.total) + ", '" + order.status + "')";
        db.execute(query);
    }

    Order getById(int id) override {
        std::string query = "SELECT * FROM orders WHERE id = " + std::to_string(id);
        auto result = db.query(query);
        return parseOrderFromResult(result);
    }
};
// ...continues on next slide
```

📌 3. Adapters (Implementations) (2/2)

[Code continues across slides]

```
// ...continued from previous slide

// Secondary Adapter: In-Memory Repository (for testing!)
class InMemoryOrderRepository : public IOrderRepository {
    std::map<int, Order> orders;
    int nextId = 1;

public:
    void save(const Order& order) override {
        orders[nextId++] = order;
    }

    Order getById(int id) override {
        return orders[id];
    }
};
```

topic_7_clean_hexagonal_architecture

Theory

📌 Flow: Request to Response (1/2)

```
1. HTTP Request arrives
        ↓
2. Primary Adapter (REST Controller)
   Translates HTTP → Domain Objects
        ↓
3. Primary Port (IOrderService interface)
        ↓
4. Business Logic (OrderService implementation)
   Validates, applies business rules
        ↓
5. Secondary Port (IOrderRepository interface)
        ↓
6. Secondary Adapter (MySQLOrderRepository)
   Saves to MySQL database
        ↓
7. Returns result through layers
        ↓
8. HTTP Response sent
```

Example Code:

```
// 1. HTTP Request
HttpRequest request = {
    method: "POST",
    path: "/orders",
    body: "{\"total\": 99.99}"
};

// 2. Primary Adapter
OrderController controller(orderService);
HttpResponse response = controller.handlePlaceOrderRequest(request);
    // Internally:
    // Order order = parseJSON(request.body);
    // orderService->placeOrder(order);  // ← Calls business logic

// 3-4. Business Logic (OrderService)
void OrderService::placeOrder(Order order) {
    // Validate business rules
    if (order.total <= 0) throw std::invalid_argument("Invalid total");
// ...continues on next slide
```

📌 Flow: Request to Response (2/2)

[Code continues across slides]

```
// ...continued from previous slide

    // Save via secondary port
    repository->save(order);  // ← Calls adapter
}

// 5-6. Secondary Adapter
void MySQLOrderRepository::save(const Order& order) {
    db.execute("INSERT INTO orders ...");
}

// 7-8. Response
HttpResponse{200, "{\"status\": \"success\"}"}
```

topic_7_clean_hexagonal_architecture

Theory

📌 Hexagonal Architecture Benefits

Benefit	Explanation
Testability	Test business logic with mocks/in-memory adapters
Independence	Business logic doesn't depend on frameworks
Flexibility	Swap adapters (MySQL → Postgres, REST → gRPC)
Maintainability	Changes to infrastructure don't affect business logic
Parallel Development	Teams work on core logic vs adapters independently

---

topic_7_clean_hexagonal_architecture

Theory

📌 Hexagonal vs Layered Architecture

Aspect	Layered	Hexagonal
Dependency Direction	Top → Bottom (UI → Business → Data)	Outside → Inside (Adapters → Core)
Core Dependency	Business depends on Data layer	Business depends on **interfaces**
Database	Business layer imports database code	Database adapter implements interface
Testability	Need database for tests	Use in-memory adapter for tests
Framework Coupling	Often tightly coupled	Framework-independent

---

topic_7_clean_hexagonal_architecture

Theory

📌 Clean Architecture (Uncle Bob)

Clean Architecture is a generalization of Hexagonal Architecture by Robert C. Martin ("Uncle Bob").

Layers (Circles):

```
┌─────────────────────────────────────────┐
│  Frameworks & Drivers (Outer)           │ ← Web, DB, UI
│   ┌─────────────────────────────────┐   │
│   │  Interface Adapters (Middle)    │   │ ← Controllers, Presenters, Gateways
│   │   ┌─────────────────────────┐   │   │
│   │   │  Use Cases (Inner)      │   │   │ ← Business Logic
│   │   │   ┌─────────────────┐   │   │   │
│   │   │   │  Entities (Core)│   │   │   │ ← Domain Models
│   │   │   └─────────────────┘   │   │   │
│   │   └─────────────────────────┘   │   │
│   └─────────────────────────────────┘   │
└─────────────────────────────────────────┘
```

Dependency Rule: **Outer layers depend on inner layers, NEVER the reverse.**

```
Frameworks → Adapters → Use Cases → Entities
(outer)                             (inner)
```

---

topic_7_clean_hexagonal_architecture

Theory

📌 When to Use Hexagonal Architecture

Use When	Don't Use When
✅ Complex business logic	❌ Simple CRUD app
✅ Long-lived project (years)	❌ Prototype/throwaway code
✅ Need to swap implementations (DB, APIs)	❌ Fixed technology stack
✅ High testability required	❌ Small project (< 1000 LOC)
✅ Multiple interfaces (Web + Mobile + API)	❌ Tight deadline (overhead)

topic_7_clean_hexagonal_architecture

Theory

📌 Real-World Examples

Banking System:

```
Core: Account, Transaction, Balance business rules
Ports: IAccountRepository, ITransactionService
Adapters:
  - Primary: REST API, Mobile App, ATM Interface
  - Secondary: Oracle Database, SWIFT Payment Gateway
```

E-Commerce:

```
Core: Order, Product, Checkout business logic
Ports: IOrderRepository, IPaymentGateway, IEmailService
Adapters:
  - Primary: Web UI (React), Mobile App (Flutter)
  - Secondary: PostgreSQL, Stripe API, SendGrid Email
```

---

topic_7_clean_hexagonal_architecture

Edge Cases

📌 1. Anemic Domain Model (Anti-Pattern) (1/3)

Problem: Domain models are just data containers with no behavior. All logic in services.

Naive Code:

```
// ❌ Anemic Domain Model
struct Order {
    int id;
    double total;
    std::string status;
    // No methods! Just data!
};

// All logic in service
class OrderService {
    void cancelOrder(Order& order) {
        // Business logic in service, not in domain model
        if (order.status == "completed") {
            throw std::logic_error("Can't cancel completed order");
        }
        order.status = "cancelled";
    }
};
```

Why It's Bad:
- **Not object-oriented**: Objects should have behavior, not just data
- **Logic scattered**: Business rules spread across many services
- **Hard to maintain**: No single source of truth

topic_7_clean_hexagonal_architecture

Edge Cases

📌 1. Anemic Domain Model (Anti-Pattern) (2/3)

Solution: Rich Domain Model

```
// ✅ Rich Domain Model
class Order {
    int id;
    double total;
    std::string status;

public:
    // Business logic IN the domain model
    void cancel() {
        if (status == "completed") {
            throw std::logic_error("Can't cancel completed order");
        }
        status = "cancelled";
    }

    void confirm() {
        if (status != "pending") {
            throw std::logic_error("Can only confirm pending orders");
        }
        status = "completed";
    }

    bool canBeCancelled() const {
        return status == "pending" || status == "processing";
    }

    // Encapsulate state
    std::string getStatus() const { return status; }
};

// Service is thin, delegates to domain model
class OrderService {
    IOrderRepository* repository;

public:
    void cancelOrder(int orderId) {
        Order order = repository->getById(orderId);
        order.cancel();  // ← Domain model handles business logic
        repository->save(order);
// ...continues on next slide
```

📌 1. Anemic Domain Model (Anti-Pattern) (3/3)

[Code continues across slides]

```
// ...continued from previous slide
    }
};
```

Lesson: Put business logic **in domain models**, not services. Services coordinate, models encapsulate behavior.

topic_7_clean_hexagonal_architecture

Edge Cases

📌 2. Port Proliferation (Too Many Interfaces) (1/2)

Problem: Creating an interface for EVERY class leads to unnecessary complexity.

Naive Code:

```
// ❌ Interface for everything
class IOrderValidator {
    virtual bool validate(const Order& order) = 0;
};

class IOrderPriceCalculator {
    virtual double calculate(const Order& order) = 0;
};

class IOrderStatusChecker {
    virtual bool canConfirm(const Order& order) = 0;
};

// ... 50 more interfaces!
```

Why It's Bad:
- **Over-engineering**: Too many abstractions
- **Hard to understand**: Need to track many interfaces
- **No real benefit**: If only one implementation, interface adds no value

topic_7_clean_hexagonal_architecture

Edge Cases

📌 2. Port Proliferation (Too Many Interfaces) (2/2)

Solution: Create Ports Only for External Dependencies

```
// ✅ Interfaces ONLY for external dependencies
class IOrderRepository {  // ← External (database)
    virtual void save(const Order& order) = 0;
    virtual Order getById(int id) = 0;
};

class IPaymentGateway {  // ← External (payment service)
    virtual void charge(const PaymentInfo& payment, double amount) = 0;
};

// Internal logic: NO interface needed
class OrderValidator {  // ← Internal, no interface
    bool validate(const Order& order) {
        return order.getTotal() > 0 && !order.getItems().empty();
    }
};
```

Rule of Thumb:
- **Create interface** if:
- External dependency (database, API, file system)
- Multiple implementations (MySQL, Postgres, InMemory)
- Need to mock for testing

- **Skip interface** if:
- Pure business logic
- Only one implementation
- Internal utility

Lesson: **Don't over-abstract.** Create ports for external dependencies, not internal logic.

---

topic_7_clean_hexagonal_architecture

Edge Cases

📌 3. Adapter Leaking to Core (1/2)

Problem: Domain models depend on adapter-specific types (e.g., DTO classes).

Naive Code:

```
// DTO from Web Framework
struct UserDTO {  // ← Part of web adapter
    std::string json_name;  // JSON field
    std::string json_email;
};

// ❌ Domain model depends on adapter type!
class User {
    UserDTO dto;  // ← WRONG! Core depends on adapter

public:
    std::string getName() { return dto.json_name; }
};
```

Why It's Bad:
- **Coupling**: Core depends on web framework
- **Can't test**: Need web framework to test domain
- **Can't swap**: Changing web framework requires changing domain

topic_7_clean_hexagonal_architecture

Edge Cases

📌 3. Adapter Leaking to Core (2/2)

Solution: Use Domain Types, Transform in Adapter

```
// ✅ Domain model uses pure domain types
class User {
    std::string name;
    std::string email;

public:
    User(const std::string& n, const std::string& e) : name(n), email(e) {}

    std::string getName() const { return name; }
    std::string getEmail() const { return email; }
};

// Adapter transforms DTO → Domain
class UserController {
    User toDomain(const UserDTO& dto) {
        return User(dto.json_name, dto.json_email);
    }

    void handleCreateUser(HttpRequest request) {
        UserDTO dto = parseJSON(request.body);
        User user = toDomain(dto);  // ← Transform in adapter!

        userService->createUser(user);
    }
};
```

Lesson: **Core should NEVER depend on adapter types.** Transform in adapters.

---

topic_7_clean_hexagonal_architecture

Edge Cases

📌 4. Database Entities vs Domain Entities (1/3)

Problem: Using ORM entities (with annotations) as domain models.

Naive Code:

```
// ❌ Domain model with ORM annotations
@Entity  // ← ORM annotation
@Table(name = "users")
class User {
    @Id  // ← ORM annotation
    @GeneratedValue
    int id;

    @Column(name = "user_name")
    std::string name;

    // Domain logic mixed with ORM concerns
    void changeName(const std::string& newName) {
        name = newName;
    }
};
```

Why It's Bad:
- **Coupling**: Domain depends on ORM framework
- **Pollution**: Domain model cluttered with annotations
- **Hard to test**: Need ORM to test domain logic

📌 4. Database Entities vs Domain Entities (2/3)

Solution: Separate Domain Model from Persistence Model

```
// ✅ Pure Domain Model (no annotations)
class User {
    int id;
    std::string name;
    std::string email;

public:
    User(int id, const std::string& name, const std::string& email)
        : id(id), name(name), email(email) {}

    void changeName(const std::string& newName) {
        if (newName.empty()) {
            throw std::invalid_argument("Name cannot be empty");
        }
        name = newName;
    }

    int getId() const { return id; }
    std::string getName() const { return name; }
    std::string getEmail() const { return email; }
};

// Persistence Model (in adapter)
struct UserEntity {  // ← ORM entity
    int id;
    std::string db_name;  // Database column name
    std::string db_email;
};

// Adapter maps between domain and persistence
class UserRepository : public IUserRepository {
    User toDomain(const UserEntity& entity) {
        return User(entity.id, entity.db_name, entity.db_email);
    }

    UserEntity toEntity(const User& user) {
        UserEntity entity;
        entity.id = user.getId();
        entity.db_name = user.getName();
// ...continues on next slide
```

📌 4. Database Entities vs Domain Entities (3/3)

[Code continues across slides]

```
// ...continued from previous slide
        entity.db_email = user.getEmail();
        return entity;
    }

    User getById(int id) override {
        UserEntity entity = orm.find<UserEntity>(id);
        return toDomain(entity);  // ← Map to domain
    }

    void save(const User& user) override {
        UserEntity entity = toEntity(user);  // ← Map to persistence
        orm.save(entity);
    }
};
```

Lesson: **Separate domain models from persistence models.** Map in adapter.

topic_7_clean_hexagonal_architecture

Edge Cases

📌 5. Too Many Layers (Over-Engineering) (1/2)

Problem: Adding too many abstraction layers makes code hard to follow.

Naive Code:

```
// ❌ Too many layers
Controller → UseCase → Service → Domain → Repository → ORM → Database

// For a simple operation:
userController.getUser(id)
    → getUserUseCase.execute(id)
        → userService.findUser(id)
            → userDomain.load(id)
                → userRepository.get(id)
                    → ormMapper.query(id)
                        → database.select(id)

// 7 layers for one query!
```

Why It's Bad:
- **Complexity**: Hard to trace execution flow
- **Verbose**: Simple operations require many classes
- **Performance**: Many function calls

Solution: Balance Abstraction with Simplicity

```
// ✅ Pragmatic layering
Controller → Service (Use Case) → Repository → Database

userController.getUser(id)
    → userService.getUser(id)  // ← Combines use case + domain logic
        → userRepository.getById(id)
            → database.query(id)

// 3 layers - clear and maintainable
```

topic_7_clean_hexagonal_architecture

Edge Cases

📌 5. Too Many Layers (Over-Engineering) (2/2)

Guidelines:
- Use **3-4 layers** for most applications:
- Controller (Primary Adapter)
- Service (Business Logic + Use Cases)
- Repository (Secondary Adapter)
- Database

- Add more layers ONLY if needed:
- Separate Use Case layer if use cases are complex
- Separate Domain layer if rich domain models

Lesson: **Don't over-engineer.** Use Hexagonal principles without excessive layering.

📌 6. Testing with Mocks vs Real Adapters (1/2)

Problem: Should tests use mocks or real adapters (e.g., in-memory database)?

Approach 1: Mocks (Fast, Isolated)

```
// Mock Repository
class MockUserRepository : public IUserRepository {
public:
    bool saveCalled = false;
    User savedUser;

    void save(const User& user) override {
        saveCalled = true;
        savedUser = user;
    }

    User getById(int id) override {
        return User(1, "Alice", "alice@example.com");
    }
};

// Test
void testCreateUser() {
    MockUserRepository mockRepo;
    UserService service(&mockRepo);

    service.createUser("Bob", "bob@example.com");

    assert(mockRepo.saveCalled);  // ✅ Verify mock was called
    assert(mockRepo.savedUser.getName() == "Bob");
}
```

Pros: Fast, isolated, no external dependencies

Cons: Tests mock behavior, not real implementation

📌 6. Testing with Mocks vs Real Adapters (2/2)

Approach 2: Real In-Memory Adapter (Integration-Like)

```
// In-Memory Repository (real implementation)
class InMemoryUserRepository : public IUserRepository {
    std::map<int, User> users;
    int nextId = 1;

public:
    void save(const User& user) override {
        users[nextId++] = user;
    }

    User getById(int id) override {
        return users.at(id);
    }
};

// Test
void testCreateUser() {
    InMemoryUserRepository realRepo;
    UserService service(&realRepo);

    service.createUser("Bob", "bob@example.com");

    User retrieved = realRepo.getById(1);
    assert(retrieved.getName() == "Bob");  // ✅ Verify actual save/retrieve
}
```

Pros: Tests real behavior, catches more bugs
Cons: Slower, more complex

Best Practice: **Use BOTH**

1. **Unit Tests**: Use mocks for fast, isolated tests
2. **Integration Tests**: Use in-memory adapters for end-to-end tests

Lesson: **Test business logic with mocks, integration with real adapters.**

---

topic_7_clean_hexagonal_architecture

Code Examples

📌 1. Simple Hexagonal Architecture (1/6)

Goal: Implement Order Service with hexagonal architecture.

Output:

```
#include <iostream>
#include <string>
#include <map>
#include <memory>

// ═══════════════════════════════════════════════════════════
//  DOMAIN MODEL (Core)
// ═══════════════════════════════════════════════════════════

class Order {
    int id;
    double total;
    std::string status;  // "pending", "completed", "cancelled"

public:
    Order(int id, double total) : id(id), total(total), status("pending") {}

    // Business logic in domain model
    void confirm() {
        if (status != "pending") {
            throw std::logic_error("Can only confirm pending orders");
        }
        status = "completed";
    }

    void cancel() {
        if (status == "completed") {
            throw std::logic_error("Can't cancel completed order");
        }
        status = "cancelled";
    }

    // Getters
    int getId() const { return id; }
    double getTotal() const { return total; }
    std::string getStatus() const { return status; }
// ...continues on next slide
```

topic_7_clean_hexagonal_architecture

Code Examples

📌 1. Simple Hexagonal Architecture (2/6)

```
// ...continued from previous slide
};

// ═══════════════════════════════════════════════════════════════
//  SECONDARY PORTS (Interfaces for external dependencies)
// ═══════════════════════════════════════════════════════════════

class IOrderRepository {
public:
    virtual void save(const Order& order) = 0;
    virtual Order getById(int id) = 0;
    virtual ~IOrderRepository() = default;
};

class IPaymentGateway {
public:
    virtual void charge(double amount) = 0;
    virtual ~IPaymentGateway() = default;
};

// ═══════════════════════════════════════════════════════════════
//  BUSINESS LOGIC (Core)
// ═══════════════════════════════════════════════════════════════

class OrderService {
    IOrderRepository* repository;
    IPaymentGateway* paymentGateway;

public:
    OrderService(IOrderRepository* repo, IPaymentGateway* payment)
        : repository(repo), paymentGateway(payment) {}

    void placeOrder(int orderId, double total) {
        std::cout << "\n[CORE] Placing order...\n";

        // Business rule: Validate total
        if (total <= 0) {
            throw std::invalid_argument("Order total must be positive");
        }

        // Create order
// ...continues on next slide
```

topic_7_clean_hexagonal_architecture

Code Examples

📌 1. Simple Hexagonal Architecture (3/6)

```
// ...continued from previous slide
        Order order(orderId, total);

        // Business rule: Charge payment
        std::cout << "[CORE] Charging payment: $" << total << "\n";
        paymentGateway->charge(total);

        // Business rule: Confirm order
        order.confirm();

        // Save order
        repository->save(order);

        std::cout << "[CORE] Order placed successfully! Status: " << order.getStatus() << "\n";
    }

    void cancelOrder(int orderId) {
        std::cout << "\n[CORE] Cancelling order...\n";

        Order order = repository->getById(orderId);
        order.cancel();  // ← Business logic in domain model
        repository->save(order);

        std::cout << "[CORE] Order cancelled. Status: " << order.getStatus() << "\n";
    }
};

// ═══════════════════════════════════════════════════════════
//  SECONDARY ADAPTERS (Implementations)
// ═══════════════════════════════════════════════════════════

class InMemoryOrderRepository : public IOrderRepository {
    std::map<int, Order> orders;

public:
    void save(const Order& order) override {
        std::cout << "  [ADAPTER] Saving order ID " << order.getId() << " to database\n";
        orders[order.getId()] = order;
    }

    Order getById(int id) override {
// ...continues on next slide
```

topic_7_clean_hexagonal_architecture

Code Examples

📌 1. Simple Hexagonal Architecture (4/6)

```
// ...continued from previous slide
        std::cout << "  [ADAPTER] Loading order ID " << id << " from database\n";
        return orders.at(id);
    }
};

class MockPaymentGateway : public IPaymentGateway {
public:
    void charge(double amount) override {
        std::cout << "  [ADAPTER] Charging $" << amount << " via payment gateway\n";
        // Simulate payment processing
    }
};

// ═══════════════════════════════════════════════════════════════
//  PRIMARY ADAPTER (API Controller)
// ═══════════════════════════════════════════════════════════════

class OrderController {
    OrderService* orderService;

public:
    OrderController(OrderService* service) : orderService(service) {}

    void handlePlaceOrder(int orderId, double total) {
        std::cout << "\n╔═══════════════════════════════════════╗\n";
        std::cout << "│  HTTP POST /orders                    │\n";
        std::cout << "╚═══════════════════════════════════════╝\n";

        try {
            orderService->placeOrder(orderId, total);
            std::cout << "\n[CONTROLLER] HTTP 200 OK: Order created\n";
        } catch (std::exception& e) {
            std::cout << "\n[CONTROLLER] HTTP 400 Error: " << e.what() << "\n";
        }
    }

    void handleCancelOrder(int orderId) {
        std::cout << "\n╔═══════════════════════════════════════╗\n";
        std::cout << "│  HTTP DELETE /orders/" << orderId << "              │\n";
        std::cout << "╚═══════════════════════════════════════╝\n";
// ...continues on next slide
```

topic_7_clean_hexagonal_architecture

Code Examples

📌 1. Simple Hexagonal Architecture (5/6)

```
// ...continued from previous slide

        try {
            orderService->cancelOrder(orderId);
            std::cout << "\n[CONTROLLER] HTTP 200 OK: Order cancelled\n";
        } catch (std::exception& e) {
            std::cout << "\n[CONTROLLER] HTTP 400 Error: " << e.what() << "\n";
        }
    }
};

// ═══════════════════════════════════════════════════════════════
//  MAIN (Application Composition Root)
// ═══════════════════════════════════════════════════════════════

int main() {
    std::cout << "═══════════════════════════════════════════════════════\n";
    std::cout << "  Hexagonal Architecture Example                      \n";
    std::cout << "═══════════════════════════════════════════════════════\n";

    // Create adapters
    InMemoryOrderRepository repository;
    MockPaymentGateway paymentGateway;

    // Inject dependencies into core
    OrderService orderService(&repository, &paymentGateway);

    // Create primary adapter (controller)
    OrderController controller(&orderService);

    // Simulate HTTP requests
    controller.handlePlaceOrder(1, 99.99);
    controller.handleCancelOrder(1);

    std::cout << "\n═══════════════════════════════════════════════════════\n";
    std::cout << "  KEY POINTS:                                          \n";
    std::cout << "  1. Core (OrderService) depends on interfaces         \n";
    std::cout << "  2. Adapters implement interfaces                     \n";
    std::cout << "  3. Can swap adapters (InMemory → MySQL)              \n";
    std::cout << "  4. Can test with mocks (no database needed)          \n";
    std::cout << "═══════════════════════════════════════════════════════\n";
// ...continues on next slide
```

topic_7_clean_hexagonal_architecture

Code Examples

📌 1. Simple Hexagonal Architecture (6/6)

[Code continues across slides]

```
// ...continued from previous slide

    return 0;
}
```

```
['═══════════════════════════════════════════════════════════\n  Hexagonal Architecture Example\n═══════════════════════════
```

What You Learned

Key Takeaways

- ✓ What is Hexagonal Architecture?
- ✓ Simple Explanation: Power Outlet Analogy
- ✓ The Problem with Traditional Layered Architecture
- ✓ Hexagonal Architecture Solution

→ Swipe to learn →